

4.5-EDA-physical-layout

1 EDA - Sheets & Physical Layout Visualisation

```
[1]: import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_parquet("../data/raw/Waveform_tuning.parquet")

col_missing = df.isna().mean()
empty_cols = col_missing[col_missing == 1.0]

df = df.drop(columns=empty_cols.index)

pd.set_option('display.max_rows', None)
```

```
[2]: sheet_id = 980301
sheet_df = df[df["SheetId#"] == sheet_id].copy()

sheet_df.to_csv("sheet_980301_full.csv", index=False)
```

Here I exported a single sheet into csv format in order to see and have a feel of how the data is structured.

```
[3]: cols = ["SheetId#", "Color$", "HeadIdx#", "V", "F_r", "dt2", "Coverage#"]
sheet_df[cols].drop_duplicates().to_csv("sheet_978175_structure.csv",
    ↪index=False)
```

```
[4]: cols = ["Color$", "HeadIdx#", "V", "F_r", "dt2", "Coverage#"]

unique_vals = {
    col: sorted(sheet_df[col].dropna().unique())
    for col in cols
}

for k, v in unique_vals.items():
    print(f"{k} ({len(v)} values): {v}")
```

```
Color$ (1 values): ['C']
HeadIdx# (30 values): [np.int64(1), np.int64(2), np.int64(3), np.int64(4),
np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.int64(10),
```

```

np.int64(11), np.int64(12), np.int64(13), np.int64(14), np.int64(15),
np.int64(16), np.int64(17), np.int64(18), np.int64(19), np.int64(20),
np.int64(21), np.int64(22), np.int64(23), np.int64(24), np.int64(25),
np.int64(26), np.int64(27), np.int64(28), np.int64(29), np.int64(30)]
V (1 values): [np.float64(20.0)]
F_r (1 values): [np.float64(1.02)]
dt2 (5 values): [np.float64(-1100.0), np.float64(-900.0), np.float64(-700.0),
np.float64(-500.0), np.float64(-300.0)]
Coverage# (30 values): [np.float64(2.0), np.float64(3.0), np.float64(4.0),
np.float64(5.0), np.float64(6.0), np.float64(7.0), np.float64(8.0),
np.float64(9.0), np.float64(10.0), np.float64(11.0), np.float64(12.0),
np.float64(13.0), np.float64(14.0), np.float64(15.0), np.float64(16.0),
np.float64(17.0), np.float64(18.0), np.float64(19.0), np.float64(20.0),
np.float64(21.0), np.float64(22.0), np.float64(23.0), np.float64(24.0),
np.float64(25.0), np.float64(26.0), np.float64(27.0), np.float64(28.0),
np.float64(29.0), np.float64(30.0), np.float64(31.0)]

```

Start slow: pick ONE fixed condition (Color, V, F_r, dt2, Coverage) and reconstruct the full 33,600-nozzle spatial profile across all 6 sheet repeats.

Goal: verify the reconstruction looks sensible before scaling up.

```

[5]: FIXED = dict(
    Color    = "C",
    V        = 20.0,
    F_r      = 1.02,
    dt2      = -1100.0,
    Coverage = 2.0,
)

N_CHIPS  = 30
N_NOZZLE = 1120
N_TOTAL  = N_CHIPS * N_NOZZLE  # 33,600

NOZZLE_COLS = [f"Value_{i:03d}#" for i in range(N_NOZZLE)]

df.columns = df.columns.str.strip()

# Normalize
df["Color"]    = df["Color$"].str.strip()
df["SheetId"]  = df["SheetId#"].astype(int)
df["HeadIdx"]  = df["HeadIdx#"].astype(int)
df["Coverage"] = df["Coverage#"].astype(float)

```

```

[6]: mask = (
    (df["Color"]    == FIXED["Color"]) &
    (df["V"]        == FIXED["V"])    &
    (df["F_r"]      == FIXED["F_r"])  &

```

```

(df["dt2"] == FIXED["dt2"]) &
(df["Coverage"] == FIXED["Coverage"])
)
sub = df[mask].copy()

print(f"\nFiltered to condition: {FIXED}")
print(f"Rows found: {len(sub)}")
print(f"Unique SheetIds: {sorted(sub['SheetId'].unique())}")
print(f"Unique HeadIdx values: {sorted(sub['HeadIdx'].unique())}")

```

```

Filtered to condition: {'Color': 'C', 'V': 20.0, 'F_r': 1.02, 'dt2': -1100.0,
'Coverage': 2.0}
Rows found: 179
Unique SheetIds: [np.int64(980301), np.int64(980302), np.int64(980303),
np.int64(980304), np.int64(980305), np.int64(980306)]
Unique HeadIdx values: [np.int64(1), np.int64(2), np.int64(3), np.int64(4),
np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.int64(10),
np.int64(11), np.int64(12), np.int64(13), np.int64(14), np.int64(15),
np.int64(16), np.int64(17), np.int64(18), np.int64(19), np.int64(20),
np.int64(21), np.int64(22), np.int64(23), np.int64(24), np.int64(25),
np.int64(26), np.int64(27), np.int64(28), np.int64(29), np.int64(30)]

```

```

[7]: import numpy as np

def reconstruct_profile(group_df):
    sorted_df = group_df.sort_values("HeadIdx")

    # Verify all 30 chips present
    heads = sorted_df["HeadIdx"].tolist()
    if heads != list(range(1, N_CHIPS + 1)):
        print(f" WARNING: HeadIdx sequence unexpected: {heads}")

    return np.concatenate(
        [sorted_df.loc[idx, NOZZLE_COLS].values for idx in sorted_df.index]
    )

profiles = {}
for sheet_id, grp in sub.groupby("SheetId"):
    profiles[sheet_id] = reconstruct_profile(grp)
    print(f" Sheet {sheet_id}: profile shape={profiles[sheet_id].shape}, "
          f"mean={profiles[sheet_id].mean():.4f}, std={profiles[sheet_id].std():"
          f"↵.4f}")

sheet_ids = sorted(profiles.keys())

```

```

print("\n Per-sheet summary")
for sid in sheet_ids:
    p = profiles[sid]
    print(f" Sheet {sid}: len={len(p)}, mean={p.mean():.4f}, "
          f"std={p.std():.4f}, min={p.min():.4f}, max={p.max():.4f}")

```

```

Sheet 980301: profile shape=(33600,), mean=0.1003, std=0.0101
Sheet 980302: profile shape=(33600,), mean=0.0994, std=0.0101
Sheet 980303: profile shape=(33600,), mean=0.0994, std=0.0100
Sheet 980304: profile shape=(33600,), mean=0.0989, std=0.0102
Sheet 980305: profile shape=(33600,), mean=0.0990, std=0.0100
WARNING: HeadIdx sequence unexpected: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 27, 28, 29, 30]
Sheet 980306: profile shape=(32480,), mean=0.0993, std=0.0101

```

Per-sheet summary

```

Sheet 980301: len=33600, mean=0.1003, std=0.0101, min=0.0617, max=0.1486
Sheet 980302: len=33600, mean=0.0994, std=0.0101, min=0.0588, max=0.1481
Sheet 980303: len=33600, mean=0.0994, std=0.0100, min=0.0582, max=0.1512
Sheet 980304: len=33600, mean=0.0989, std=0.0102, min=0.0576, max=0.1279
Sheet 980305: len=33600, mean=0.0990, std=0.0100, min=0.0571, max=0.1241
Sheet 980306: len=32480, mean=0.0993, std=0.0101, min=0.0570, max=0.1316

```

```

[8]: fig, axes = plt.subplots(2, 1, figsize=(18, 10))
fig.suptitle(
    f"Spatial SD Profile - Color={FIXED['Color']}, V={FIXED['V']}, "
    f"F_r={FIXED['F_r']}, dt2={FIXED['dt2']}, Coverage={FIXED['Coverage']}\n"
    f"Full 33,600-nozzle reconstruction (30 chips × 1120 nozzles)",
    fontsize=12, fontweight="bold"
)

complete = {sid: p for sid, p in profiles.items() if len(p) == N_TOTAL}
incomplete = {sid: p for sid, p in profiles.items() if len(p) != N_TOTAL}

ax = axes[0]
for sid, p in complete.items():
    ax.plot(np.arange(len(p)), p, lw=0.4, alpha=0.6, label=f"Sheet {sid}")
for sid, p in incomplete.items():
    ax.plot(np.arange(len(p)), p, lw=0.4, alpha=0.6, linestyle=":",
    label=f"Sheet {sid} incomplete")

for chip in range(1, N_CHIPS):
    ax.axvline(chip * N_NOZZLE, color="gray", lw=0.5, linestyle="--", alpha=0.4)
ax.set_xlabel("Nozzle index (physical position across sheet)")
ax.set_ylabel("Scanner Density")

```

```

ax.set_title(f"All sheet repeats overlaid ({len(incomplete)} incomplete,
↳flagged with )")
ax.legend(fontsize=7, ncol=6)
ax.grid(alpha=0.2)

ax2 = ax.twinx()
ax2.set_xlim(ax.get_xlim())
chip_centers = [(i + 0.5) * N_NOZZLE for i in range(N_CHIPS)]
ax2.set_xticks(chip_centers)
ax2.set_xticklabels([str(i+1) for i in range(N_CHIPS)], fontsize=6)
ax2.set_xlabel("Chip index", fontsize=9)

x = np.arange(N_TOTAL)
profile_matrix = np.stack(list(complete.values()))
mean_profile = profile_matrix.mean(axis=0)
std_profile = profile_matrix.std(axis=0)

ax = axes[1]
ax.plot(x, mean_profile, lw=0.8, color="steelblue",
        label=f"Mean across {len(complete)} complete sheets")
ax.fill_between(x,
                mean_profile - std_profile,
                mean_profile + std_profile,
                alpha=0.3, color="steelblue", label="+1 std (noise)")

for chip in range(1, N_CHIPS):
    ax.axvline(chip * N_NOZZLE, color="gray", lw=0.5, linestyle="--", alpha=0.4)
ax.set_xlabel("Nozzle index (physical position across sheet)")
ax.set_ylabel("Scanner Density")
ax.set_title(f"Mean ± 1 std - computed on {len(complete)} complete sheets only")
ax.legend(fontsize=9)
ax.grid(alpha=0.2)

plt.tight_layout()
plt.show()

# PLOT 2: Noise floor
fig, ax = plt.subplots(figsize=(18, 4))
ax.plot(x, std_profile, lw=0.5, color="firebrick", alpha=0.8)
ax.set_xlabel("Nozzle index (physical position)")
ax.set_ylabel("Std of SD across repeats")
ax.set_title(
    f"Noise floor per nozzle - "
    f"median std={np.median(std_profile):.5f}, max={std_profile.max():.5f} "
    f"(computed on {len(complete)} complete sheets)"
)

```

```

for chip in range(1, N_CHIPS):
    ax.axvline(chip * N_NOZZLE, color="gray", lw=0.5, linestyle="--", alpha=0.4)
ax.grid(alpha=0.2)
plt.tight_layout()
plt.show()

# PLOT 3: Chip boundary zoom
fig, axes = plt.subplots(3, 3, figsize=(16, 10))
fig.suptitle("Chip boundary zoom - verifying spatial continuity", fontsize=12)

boundary_chips = np.linspace(1, N_CHIPS - 1, 9, dtype=int)

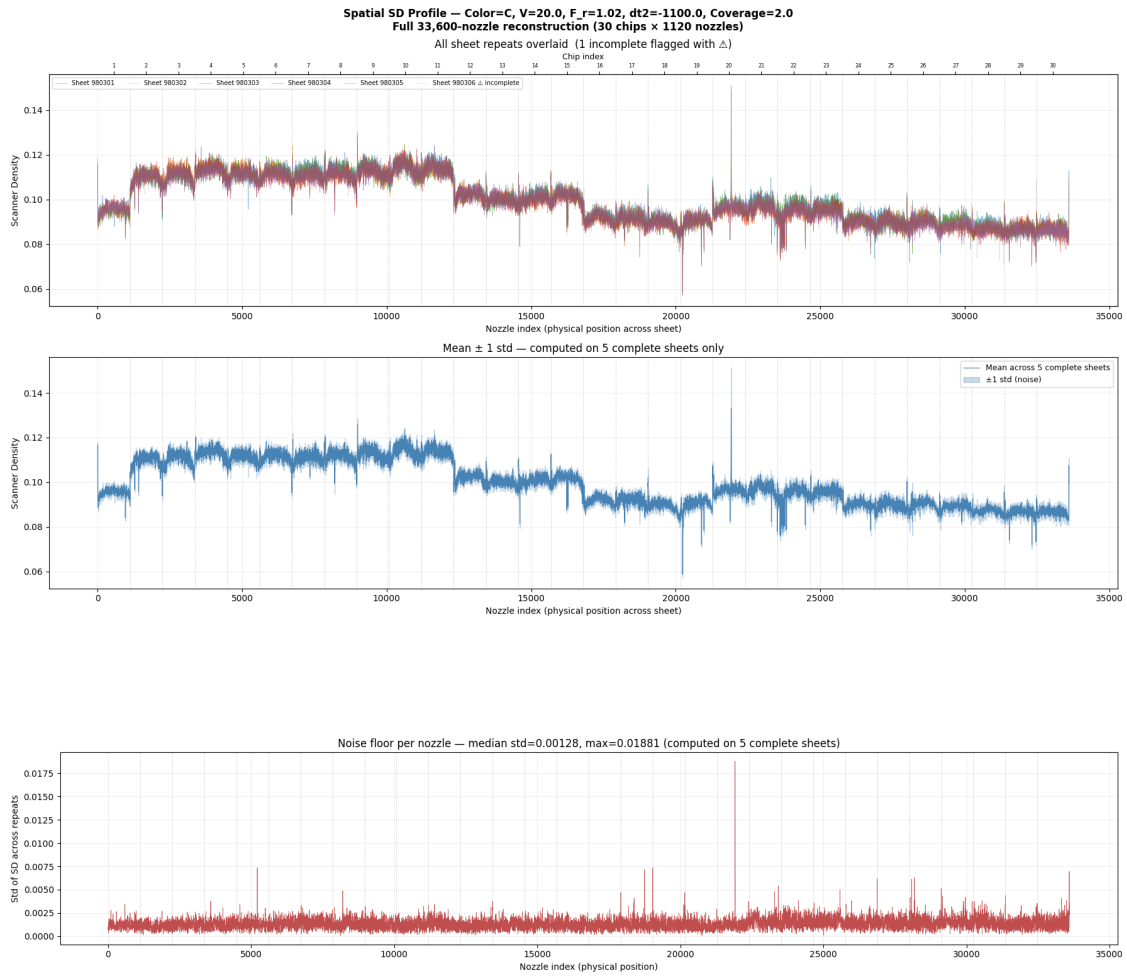
for ax, chip_idx in zip(axes.flat, boundary_chips):
    center = chip_idx * N_NOZZLE
    lo, hi = max(0, center - 80), min(N_TOTAL, center + 80)

    for sid, p in complete.items():
        ax.plot(np.arange(lo, hi), p[lo:hi], lw=0.6, alpha=0.5, label=f"{sid}")
    for sid, p in incomplete.items():
        if lo < len(p):
            hi_safe = min(hi, len(p))
            ax.plot(np.arange(lo, hi_safe), p[lo:hi_safe],
                    lw=0.6, alpha=0.5, linestyle=":", label=f"{sid} ")

    ax.axvline(center, color="red", lw=1.2, linestyle="--")
    ax.set_title(f"Chip {chip_idx} → {chip_idx+1}", fontsize=9)
    ax.set_xlabel("Nozzle index", fontsize=7)
    ax.set_ylabel("SD", fontsize=7)
    ax.tick_params(labelsize=6)

plt.tight_layout()
plt.show()

```



Chip boundary zoom — verifying spatial continuity

